



DESKTOP

Formation - Partie 4 - JAVA FX

Qu'est-ce que JavaFX ?

- **Framework moderne** : Conçu pour créer des applications desktop Java riches, performantes et cross-platform (Windows, macOS, Linux).
- **Rendu matériel accéléré** : Utilisation du pipeline graphique Prism qui s'appuie sur la carte graphique (via DirectX ou OpenGL) pour des animations fluides, à l'image du fonctionnement de WPF avec DirectX.
- **Unités de mesure indépendantes** : L'interface graphique est vectorielle et s'adapte nativement aux écrans haute densité.



Anatomie d'une application

- **Le Stage** (`javafx.stage.Stage`) : représente la fenêtre principale de l'application.
- **La Scene** (`javafx.scene.Scene`) : le conteneur physique du contenu de la fenêtre. Un Stage ne peut afficher qu'une seule Scene à la fois.
- **Le Scene Graph** : un arbre hiérarchique de composants graphiques appelés Nodes (boutons, layouts, formes géométriques). La racine de cet arbre est le Root Node de la Scene.

L'écosystème moderne

- **OpenJFX** : depuis Java 11, JavaFX est découplé du JDK standard. Il s'intègre désormais comme une bibliothèque tierce via un gestionnaire de dépendances.
- **Modularité (JPMS)** : utilisation obligatoire du fichier module-info.java pour encapsuler son code et déclarer les modules JavaFX requis :
 - requires javafx.controls;
 - requires javafx.fxml;
- **Outils de Build** : Maven ou Gradle gèrent automatiquement le téléchargement des SDK natifs selon la plateforme cible (Windows, Mac ou Linux).



Structure minimale pour débiter

```
Structure minimale

package com.ynov.app;

import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.layout.StackPane;
import javafx.stage.Stage;

public class MainApp extends Application {

    @Override
    public void start(Stage primaryStage) {
        // 1. Création d'un composant (Node)
        Label welcomeLabel = new Label("Bienvenue en JavaFX !");

        // 2. Choix du layout racine (Root Node)
        StackPane root = new StackPane(welcomeLabel);

        // 3. Initialisation de la scène avec des dimensions dynamiques
        Scene scene = new Scene(root, 800, 450);

        // 4. Configuration du Stage
        primaryStage.setTitle("Ma Première Application JavaFX");
        primaryStage.setScene(scene);
        primaryStage.show(); // Affichage à l'écran
    }

    public static void main(String[] args) {
        launch(args); // Lance le cycle de vie de l'application
    }
}
```

Le cycle de vie d'une application

- Toute application JavaFX hérite de la classe abstraite `javafx.application.Application`.
- `init()` : Exécutée en premier sur un thread d'initialisation (idéal pour charger des configurations ou des connexions de données lourdes).
- `start(Stage primaryStage)` : Point d'entrée principal de l'UI, exécuté sur le thread JavaFX Application. C'est ici que l'on construit son arbre visuel.
- `stop()` : Appelée automatiquement lors de la fermeture de l'application (idéal pour fermer proprement des services ou des bases de données).

Le fichier de configuration

```
module-info.java

module com.ynov.app {
    // Déclare l'utilisation des composants graphiques de base
    requires javafx.controls;

    // Si l'on utilise des interfaces basées sur du balisage XML
    requires javafx.fxml;

    // Permet au moteur JavaFX d'accéder à la classe principale par réflexion
    exports com.ynov.app;
}
```

Exercices

Pourquoi interdire les coordonnées fixes ?

- En développement desktop moderne, le positionnement absolu (coordonnées \$X, Y\$ fixes) est une mauvaise pratique.
- Problématiques :
 - Variabilité des résolutions d'écran (4K, HiDPI, écrans portables).
 - Redimensionnement de la fenêtre par l'utilisateur.
 - Traduction de l'application (la taille des textes varie selon la langue).
- La solution : Les conteneurs de disposition dynamiques (Layouts). Ils calculent automatiquement la taille et la position de leurs enfants en temps réel.

Les conteneurs de base (HBox & VBox)

- **HBox** : Aligne tous ses composants enfants sur une seule ligne horizontale.
- **VBox** : Aligne tous ses composants enfants sur une seule colonne verticale.
- Propriétés clés :
 - **spacing** : Espace en pixels entre chaque enfant.
 - **alignment** : Aligne le bloc d'enfants (ex: au centre, en haut à gauche).
 - **VBox.setVgrow()** / **HBox.setHgrow()** : Indique à un composant s'il doit s'étirer pour occuper l'espace restant.

Structurer une fenêtre avec BorderPane

- **BorderPane** : Idéal pour définir la structure globale d'une application.
- Il divise l'espace en 5 régions :
 - Top : Barre de menus, barre d'outils.
 - Left : Menu de navigation latéral.
 - Center : Contenu principal (prend tout l'espace restant).
 - Right : Panneau d'informations secondaires, propriétés.
 - Bottom : Barre de statut, pied de page.
- Vous n'êtes pas obligé d'utiliser les 5 zones. La zone Center s'agrandit automatiquement pour combler les zones vides.

Le GridPane (équivalent du Grid sur WPF)

- **GridPane** : Organise les composants dans une grille flexible de lignes et de colonnes.
- Très puissant pour les formulaires ou les tableaux de bord alignés.
- Contrairement à un tableau HTML rigide, on y place les éléments de manière déclarative :

```
GridPane grid = new GridPane();
grid.setHgap(10); // Espace horizontal entre les cellules
grid.setVgap(10); // Espace vertical entre les cellules

// Syntaxe : grid.add(Node child, int columnIndex, int rowIndex);
grid.add(new Label("Nom :"), 0, 0);
grid.add(new TextField(), 1, 0);
```

Marges et Alignements

- **Padding** : Espace intérieur entre les bords du conteneur et ses composants enfants :
 - `conteneur.setPadding(new Insets(15));` (marge interne de 15px sur les 4 côtés).
- **Margin** : Espace extérieur autour d'un composant spécifique par rapport à ses voisins :
 - `GridPane.setMargin(monBouton, new Insets(0, 0, 10, 0));`
- **Alignment** : Positionnement d'un élément dans son espace imparti :
 - `monLabel.setAlignment(Pos.CENTER);`

Exemple complet

```
Exemple marges et alignements

@Override
public void start(Stage primaryStage) {
    BorderPane root = new BorderPane();

    // 1. Barre supérieure
    HBox topBar = new HBox(new Label("Mon Application"));
    topBar.setPadding(new Insets(10));
    topBar.setStyle("-fx-background-color: #333; -fx-text-fill: white;");
    root.setTop(topBar);

    // 2. Formulaire central (GridPane)
    GridPane formGrid = new GridPane();
    formGrid.setPadding(new Insets(20));
    formGrid.setHgap(10); formGrid.setVgap(10);
    formGrid.setAlignment(Pos.CENTER); // Centre toute la grille

    formGrid.add(new Label("Identifiant :"), 0, 0);
    formGrid.add(new TextField(), 1, 0);
    formGrid.add(new Label("Mot de passe :"), 0, 1);
    formGrid.add(new PasswordField(), 1, 1);

    root.setCenter(formGrid);

    Scene scene = new Scene(root, 500, 350);
    primaryStage.setScene(scene);
    primaryStage.show();
}
```

Exercices

Les contrôles de base communs

- **Affichage et saisie :**

- Label : Pour afficher du texte statique non éditable.
- TextField : Champ de saisie de texte standard sur une seule ligne.
- PasswordField : Masque les caractères saisis (utile pour les mots de passe).

- **Actions et choix :**

- Button : Déclenche une action lors du clic.
- CheckBox : Case à cocher pour des choix indépendants (états : coché/décoché).
- RadioButton : Bouton radio (s'utilise avec un ToggleGroup pour un choix exclusif).

- **Listes et collections :**

- ListView<T> : Composant pour afficher une liste déroulante de données.

Gestion des évènements en code pur

- Pour interagir avec un composant, on lui attache un écouteur d'événements (Event Listener).
- La méthode la plus courante pour un bouton est `.setOnAction()`

```
Gestion des évènements

Button btnAjouter = new Button("Ajouter");
TextField inputNote = new TextField();

// Utilisation d'une expression Lambda (Java 8+)
btnAjouter.setOnAction(event -> {
    String texte = inputNote.getText(); // Récupération manuelle
    if (!texte.isBlank()) {
        System.out.println("Note ajoutée : " + texte);
        inputNote.clear(); // Nettoyage manuel de l'UI
    }
});
```

Introduction au FXML : séparer le design du code

- Un langage déclaratif basé sur le format XML utilisé pour concevoir et structurer graphiquement l'interface utilisateur.
- Avantages :
 - Séparation nette des préoccupations : Le designer/développeur UI modifie le FXML, le développeur d'application code la logique en Java.
 - Scene Builder : Logiciel WYSIWYG officiel pour concevoir vos interfaces graphiques visuellement sans toucher au code XML.
- Liaison : le fichier FXML pointe vers une classe Java appelée Contrôleur via l'attribut fx:controller.

Exemple de balisage FXML

```
view.fxml

<?xml version="1.0" encoding="UTF-8"?>
<?import javafx.scene.layout.VBox?>
<?import javafx.scene.control.TextField?>
<?import javafx.scene.control.Button?>
<?import javafx.geometry.Insets?>

<VBox xmlns="http://javafx.com/javafx" xmlns:fx="http://javafx.com/fxml"
      fx:controller="com.ynov.app.TODOController" spacing="15.0">
  <padding>
    <Insets top="20" right="20" bottom="20" left="20"/>
  </padding>

  <TextField fx:id="inputNote" promptText="Écrire une note..."/>

  <Button text="Ajouter" onAction="#onBtnAjouterClick" prefWidth="100.0"/>
</VBox>
```

Le contrôleur java

```
TodoController.java

package com.ynov.app;

import javafx.fxml.FXML;
import javafx.scene.control.TextField;

public class TodoController {

    // L'annotation @FXML injecte automatiquement l'élément défini dans le XML
    // Le nom de la variable doit être EXACTEMENT le même que le fx:id
    @FXML
    private TextField inputNote;

    // Cette méthode est appelée automatiquement lors du clic sur le bouton
    @FXML
    private void onBtnAjouterClick() {
        String texte = inputNote.getText();
        if (texte != null && !texte.isBlank()) {
            System.out.println("Ajout via FXML : " + texte);
            inputNote.clear();
        }
    }
}
```

Comment charger un fichier FXML au démarrage

- Pour afficher une interface conçue en FXML, on remplace l'instanciation manuelle des layouts par un chargeur dédié (FXMLLoader) :

```
Méthode start

@Override
public void start(Stage primaryStage) throws Exception {
    // 1. Charger le fichier FXML depuis les ressources du projet
    FXMLLoader loader = new FXMLLoader(getClass().getResource("/views/view.fxml"));
    Parent root = loader.load();

    // 2. Assigner le layout racine chargé à la scène
    Scene scene = new Scene(root, 400, 300);

    primaryStage.setTitle("Ma Todo App FXML");
    primaryStage.setScene(scene);
    primaryStage.show();
}
```

Exercices

Le problèmes des types natifs (variables standards)

- En Java classique, modifier une variable `String nom = "Alice";` n'émet aucun signal.
- Si cette variable est affichée dans un Label, l'interface graphique n'a aucun moyen de savoir qu'elle doit se rafraîchir.
- En WPF, vous devez implémenter l'interface `INotifyPropertyChanged` pour notifier la vue.
- En JavaFX, pas besoin d'interface lourde à écrire, le framework intègre nativement un système d'objets observables appelé les Properties.

Les Properties JavaFX

- Une Property est un conteneur qui enveloppe une valeur et génère des événements automatiques dès que sa valeur change.
- Les équivalences de types :
 - String : StringProperty (implémentation : SimpleStringProperty)
 - int : IntegerProperty (implémentation : SimpleIntegerProperty)
 - boolean : BooleanProperty (implémentation : SimpleBooleanProperty)

```
properties

public class Joueur {
    // 1. Déclaration de la propriété interne
    private final StringProperty nom = new SimpleStringProperty(this, "nom", "");

    // 2. Getter standard (renvoie la valeur brute)
    public String getNom() { return nom.get(); }

    // 3. Setter standard (modifie la valeur et notifie)
    public void setNom(String nouveauNom) { nom.set(nouveauNom); }

    // 4. Accès à l'objet Property (pour le binding)
    public StringProperty nomProperty() { return nom; }
}
```


Rappels : le mécanisme de data binding

- Le Data Binding consiste à lier deux propriétés pour qu'elles se synchronisent automatiquement.
- **Binding Unidirectionnel** (.bind()) : La cible écoute la source. Si la source change, la cible se met à jour. L'inverse est impossible.
- **Binding Bidirectionnel** (.bindBidirectional()) : Idéal pour les formulaires de saisie. Si l'utilisateur tape du texte, la propriété Java change. Si le code modifie la propriété Java, le texte à l'écran change tout seul.

Exemple dans le contrôleur

```
Uploaded using RayThis Extension

public class ConnexionController {
    @FXML private TextField inputEmail;
    @FXML private Button btnValider;

    // Propriété de notre modèle de données (ou ViewModel)
    private final StringProperty emailSaisi = new SimpleStringProperty("");

    @FXML
    public void initialize() {
        // 1. Liaison bidirectionnelle entre le champ de saisie et notre variable Java
        inputEmail.textProperty().bindBidirectional(emailSaisi);

        // 2. Liaison unidirectionnelle conditionnelle :
        // Le bouton est désactivé (disabled) tant que le texte est vide
        btnValider.disableProperty().bind(emailSaisi.isEmpty());
    }
}
```

Les collections observables

- Pour lier des listes d'éléments (comme des tâches ou des notes) à des composants visuels (ListView), **une simple ArrayList ne suffit pas**.
- JavaFX introduit **l'ObservableList<T>** (l'équivalent exact de **l'ObservableCollection<T>** en .NET/WPF).
- Elle intercepte les ajouts, suppressions et modifications pour actualiser instantanément l'écran visuel.

```
Uploaded using RayThis Extension

// Création d'une liste observable vide
ObservableList<String> listeTaches = FXCollections.observableArrayList();

// Liaison directe avec le composant graphique (au démarrage)
maListViewGraphique.setItems(listeTaches);

// Ensuite, on manipule uniquement la liste de données !
listeTaches.add("Acheter du pain"); // L'UI s'actualise toute seule !
```

Architecture MVVM / MVC

- **Model** : vos classes métiers pures.
- **View** : vos fichiers .fxml représentant l'agencement visuel.
- **Controller / ViewModel** : Le cerveau applicatif.
- Le contrôleur ne doit jamais manipuler directement les propriétés d'apparence des composants graphiques. À la place, il modifie des Properties et des ObservableLists. Le mécanisme de Data Binding se charge de répercuter le changement sur la Vue.

Exercices

Scene Graph Orienté Objet vs Canvas

- Pour faire du jeu vidéo ou du graphisme en JavaFX, deux approches s'affrontent
- L'approche **Vectorielle** (`javafx.scene.shape.Shape`) :
 - Chaque élément est un objet Java (Circle, Rectangle, Arc).
 - On modifie ses coordonnées (`setTranslateX()`) et le moteur se charge de le redessiner.
 - Idéal pour débiter : gestion native des clics et propriétés observables.
- L'approche **Pixel** (`javafx.scene.canvas.Canvas`) :
 - Une zone de dessin vierge où l'on peint manuellement via un contexte graphique (`GraphicsContext`).
 - Plus performant pour des milliers d'éléments, mais demande de tout recalculer soi-même.

Les Shapes clés pour Pacman

- **Le Labyrinthe** : Construit à l'aide de rectangles. Rectangle mur = `new Rectangle(width, height, Color.BLUE);`
- **Les Fantômes / Pac-gums** : Représentés par des cercles. Circle pacGum = `new Circle(radius, Color.ORANGE);`
- **Pacman** : Représenté idéalement par un arc de cercle (Arc). L'arc permet de simuler la bouche ouverte :

```
Shapes clés

Arc pacman = new Arc();
pacman.setCenterX(100);
pacman.setCenterY(100);
pacman.setRadiusX(20); pacman.setRadiusY(20);
pacman.setStartAngle(45); // Angle de départ de la bouche
pacman.setLength(270);    // Ouverture (360 - 90 degrés de bouche)
pacman.setType(ArcType.ROUND);
pacman.setFill(Color.YELLOW);
```

Capturer les entrées clavier

- Pour déplacer un personnage, l'application doit écouter en continu les touches pressées par l'utilisateur.
- On attache cet écouteur directement sur la Scene globale

```
Capturer les entrées clavier

scene.setOnKeyPressed(event -> {
    KeyCode touche = event.getCode();

    switch (touche) {
        case UP, Z    -> dirY = -5; // On monte (Y diminue vers le haut)
        case DOWN, S  -> dirY = 5;  // On descend
        case LEFT, Q   -> dirX = -5; // On va à gauche
        case RIGHT, D  -> dirX = 5;  // On va à droite
    }
});
```


La boucle de rendu : AnimationTimer

- Un jeu a besoin d'un battement de cœur régulier pour mettre à jour le monde (généralement 60 fois par seconde, soit 60 FPS).
- JavaFX fournit la classe AnimationTimer. Elle se synchronise nativement avec le taux de rafraîchissement de l'écran.

```
AnimationTimer
AnimationTimer gameLoop = new AnimationTimer() {
    @Override
    public void handle(long now) {
        // Cette méthode est appelée à chaque frame (environ toutes les 16ms)

        // 1. Mettre à jour les positions (Mouvement)
        pacman.setTranslateX(pacman.getTranslateX() + dirX);
        pacman.setTranslateY(pacman.getTranslateY() + dirY);

        // 2. Gérer la logique (Collisions, Score)
        verifierCollisions();
    }
};

gameLoop.start(); // Lance la boucle
```

Notions de base sur les Collisions 2D

- En mode vectoriel (Scene Graph), JavaFX propose des outils d'intersection natifs très simples à utiliser pour débiter
- Détection par boîte englobante (**Bounds**) :
 - Vérifie si les rectangles invisibles qui entourent vos objets se touchent.
 - `boolean collision = pacman.getBoundsInParent().intersects(fantome.getBoundsInParent());`
- Détection géométrique précise (**Shape**) :
 - Calcule l'intersection réelle des formes, pixel par pixel.

```
Uploaded using RayThis Extension

// Crée une nouvelle forme géométrique représentant la zone de contact
Shape intersection = Shape.intersect(pacman, pacGum);

if (intersection.getBoundsInLocal().getWidth() > -1) {
    // La zone d'intersection existe -> Pacman mange la pac-gum !
}
```

Exercices

TP 4 : Validation des acquis